MCP: from protocol to production



How to use, how to host, how to manage

Mehdi Labadi
Solutions Expert, Devtools, Microsoft Canada

Universe 2026 cheatsheet: what's new

Only features first announced at Universe (28 Oct), three per area. Status in brackets.

AGENTS & APP

Copilot Projects (link)
Give a goal; agents work across repos (preview).

Image generation (link)
Create and edit images in Copilot chat (preview).

Security extension (link)
Fix code scanning alerts in the app (preview).

CONTEXT & MEMORY

Copilot connectors (link)
Managed MCP servers, one sign-in (preview).

Copilot Dreaming (link)
Learns from past work; proposes PRs (preview).

Issue artifacts (link)
Attach plans and research to the issue (preview).

REVIEW & SECURITY

Code review Max (link)
Several review agents, several models (preview).

AI Scan for repos (link)
Scans the whole repo, not just PRs (preview).

AI app scanning (link)
Flags prompt injection in pull requests (preview).

PLATFORM & PLANS

GitHub Enterprise (link)
Copilot included; pay for AI credits used.

Replica (link)
Open-source AI prototyping tool (MIT).

Previews change quickly: check the changelog for status before you demo or roll out.

Sources: GitHub Changelog, 28 Oct 2026 (a link for each announcement is in the speaker notes) | GitHub Blog

The six questions, with engineering outcomes

Each section answers one question and ends with a concrete engineering outcome.

01	**What is MCP?**	Separate host, client, server, tools, resources and prompts.
02	**How does it work?**	Read versioned wire requests, results and transport behavior.
03	**MCP, REST and CLI**	Compare REST, MCP and CLI with Azure DevOps as the example.
04	**Connect in Copilot**	Connect the Azure DevOps MCP server in the Copilot CLI and app.
05	**Build your own server**	Run a Hello World lab, then add delegated sign-in for user data.
06	**Manage the registry**	Separate discovery from execution; query a catalog; enforce policy.



The contract, roles and capability surfaces.

What is MCP?

MCP: an open standard connecting AI to real systems

What MCP is, why it exists and who governs it today.

WHAT IT IS / WHY IT EXISTS

Model Context Protocol (MCP) gives AI applications a common way to discover and use external tools, data and prompts.

Instead of rebuilding a connector for every AI app, expose capabilities once through an MCP server and reuse them across compatible clients.

FROM ANTHROPIC TO OPEN GOVERNANCE

November 2024: Anthropic open-sources MCP to address fragmented AI integrations.

December 2025: MCP joins the Agentic AI Foundation (AAIF), under the Linux Foundation.

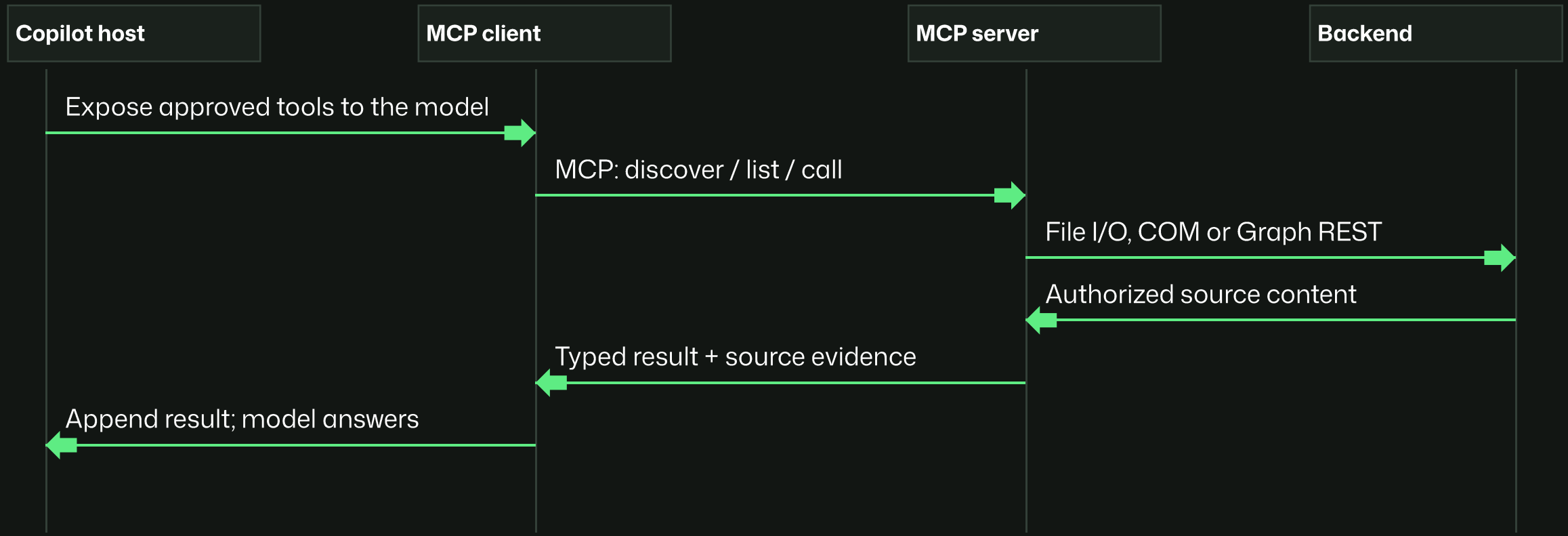
A neutral home for open, collaborative development across vendors.

Purpose: connect AI to useful context and actions, without a bespoke integration for every host.

Sources: Anthropic, Introducing MCP (Nov 2024) | Linux Foundation, AAIF announcement (9 Dec 2025)

Four components, three separate responsibilities

Read top to bottom: one tool call, from approval to the model's answer.



Host controls model/approval; SDK implements MCP; adapter enforces data access.

Tools, resources and prompts are not interchangeable

Each row: what the surface is, then how a host actually uses it.

SURFACE	WHAT IT IS	HOW IT IS USED / EXAMPLE
Tool	A typed function that reads data or performs an action.	Model requests; host dispatches tools/call. search_notes(query) finds matching lines.
Resource	Read-only content at a URI, used as context by the AI app.	Application fetches via resources/read; notes://guide returns a guide.
Prompt	A reusable, parameterized message template; not an executable tool.	User selects; host retrieves via prompts/get. review_notes(topic) builds instructions.
Optional client features	Features exposed by the client for the server to use.	Elicitation: user input. Roots: workspace URIs. Sampling: model output (deprecated in 2026-07-28).

Start with tools only; add other surfaces when they solve an actual host workflow.



Wire messages, versions, errors and transports.

How does it work?

A complete modern tools/call envelope

CALL / Client sends after tool selection and host approval

JSON-RPC REQUEST

```
{
  "jsonrpc": "2.0", "id": 42, "method": "tools/call",
  "params": {
    "name": "read_demo_note", "arguments": { "maxLines": 2 },
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientInfo": {
        "name": "workshop-client", "version": "1.0.0"
      },
      "io.modelcontextprotocol/clientCapabilities": {}
    }
  }
}
```

1. RPC envelope

id = response correlation; method = tools/call; jsonrpc = wire format.

2. Tool inputs

name selects the tool. maxLines = 2 asks for at most two lines.

3. Request context

Version and client capabilities describe protocol support, not permission.

id correlates the response; arguments are tool input; _meta is protocol context, not permission.

tools/list gives the model a usable contract

DISCOVERY / Server returns tool definitions before invocation

PUBLIC DEMO TOOL DEFINITION

```
{
  "name": "read_demo_note",
  "description": "Read the fixed public demo.txt fixture.",
  "inputSchema": {
    "type": "object", "properties": {
      "maxLines": { "type": "integer", "minimum": 1, "maximum": 20 }
    }
  },
  "annotations": {
    "readOnlyHint": true, "destructiveHint": false,
    "idempotentHint": true, "openWorldHint": false
  }
}
```

1. Tool identity

name is the callable identifier; description helps the model choose.

2. Input contract

maxLines is an integer from 1 to 20. The SDK validates the input.

3. Behavior hints

Read-only and repeatable are declared hints, not authorization or a sandbox.

Annotations are hints; schemas reject malformed values, not unauthorized data access.

Return evidence the host can display and validate

RESULT / Server replies after the handler completes

RESPONSE / SAME REQUEST ID

```
{
  "jsonrpc": "2.0", "id": 42,
  "result": {
    "resultType": "complete", "isError": false,
    "content": [{ "type": "text", "text": "Public demo note; lines 1-2." }],
    "structuredContent": {
      "path": "demo.txt",
      "lines": [
        { "line": 1, "text": "Public MCP workshop note." },
        { "line": 2, "text": "Use the SDK; never hand-roll MCP." }
      ],
      "totalLines": 3, "truncated": true }
  }
}
```

1. Match the call

id = 42 links this result to the request. isError reports tool failure.

2. Two output surfaces

content is displayable text; structuredContent holds machine-readable data.

3. Demo evidence

Line numbers identify the source. truncated means some lines were omitted.

outputSchema validates structuredContent; source IDs, line numbers and truncation prevent overclaiming.

Get it working. Keep it reliable.

Debug the connection first, then the tool. In production, watch reliability and access.

WHEN SOMETHING FAILS

Can it connect? Check the command or URL, then sign-in.

Can it call a tool? Try a small known input in MCP Inspector or a test client.

What went wrong? Read the error and logs; separate failures from valid empty results.

WHEN IT RUNS IN PRODUCTION

Monitor health: availability, response time and failed calls.

Protect access: least privilege; never log credentials or note contents.

Control changes: set timeouts, test updates and keep a rollback version.

A server being reachable does not prove its tools work. Verify one real end-to-end call.

Which type of MCP server should you use?

Two standard transports: stdio and Streamable HTTP. Local vs remote describes deployment.

TYPE / CONNECTION	PROS	CONS / TRADE-OFFS	USE CASES
Local process stdio	Simple setup; no listening port. Host launches the process. Direct access to local apps/files.	Runtime and package needed on each host. Process lifecycle tied to client. Permissions of the local account.	Personal notes and filesystem tools. Windows COM integration. Development and single-user demos.
Remote service Streamable HTTP	Shared endpoint for many clients. Central deployment and updates. No local server runtime required.	Needs hosting, authentication and per-user authorization. Network latency/availability. Streaming-aware infrastructure.	Team/enterprise services. Hosted OneNote Graph adapter. Shared business APIs and databases.
Local HTTP service Streamable HTTP	Same HTTP interface as deployment. Useful across local processes. Can outlive one client process.	Manage ports and service lifecycle. Restrict binding; validate Origin/Host. Secure access even on localhost.	Test a future hosted server. Local service shared by clients. Apps that already expose HTTP.

Local does not mean offline or private: a stdio server can call cloud APIs, and the host may use a remote model.



Three ways to reach a backend, and when MCP pays off.

MCP, REST and CLI

REST, MCP and CLI: three ways to reach a backend

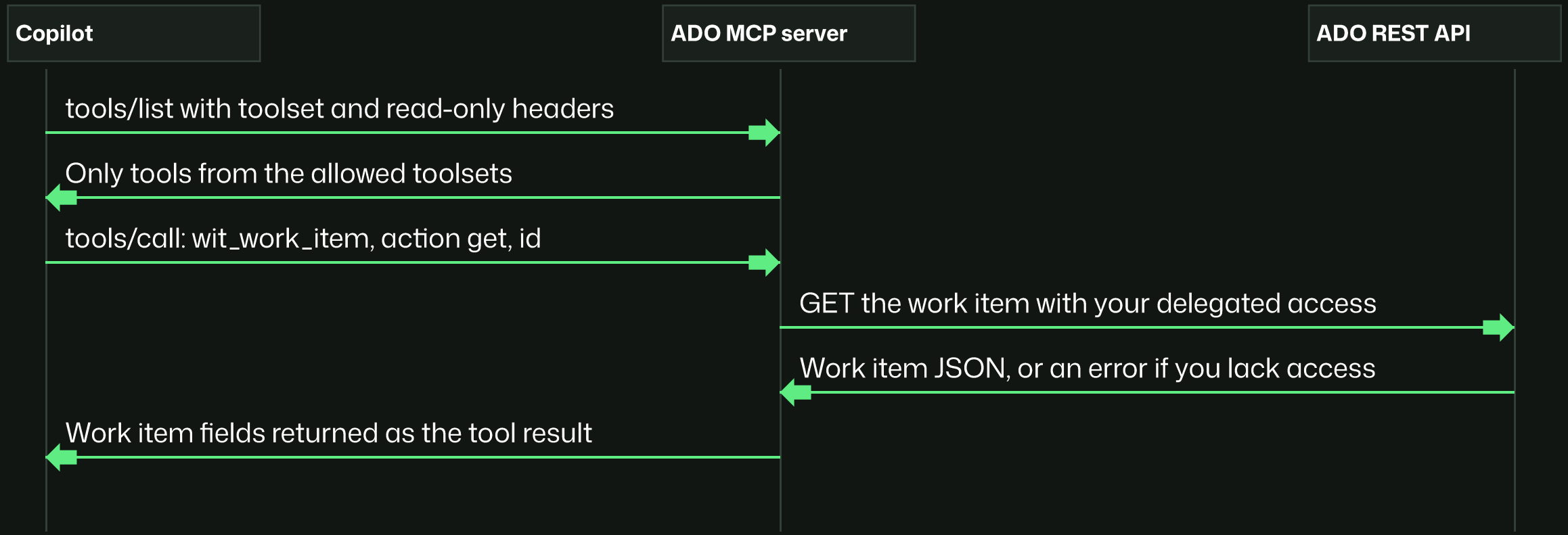
The same five dimensions, compared across all three ways in.

DIMENSION	REST / HTTP API	MCP	CLI
Contract	Resources, verbs, representations	Tools/resources/prompts; JSON-RPC methods	Commands, flags and exit codes; text or JSON output
Discovery	Docs, OpenAPI, optional hypermedia	Standard capability/list operations	--help and docs; models often know popular CLIs
Errors	HTTP status + application error body	Transport + RPC error + tool isError	Exit code + stderr message
Reuse	General applications; AI can use it too	Compatible AI hosts share a tool contract	People, scripts, CI and agents with a shell
Auth	Service-specific OAuth/ACLs	HTTP MCP auth plus backend authorization	The CLI's own local login, e.g. gh auth login

REST serves apps, a CLI serves agents with a shell, MCP serves governed tools across hosts.

Azure DevOps MCP: narrow tools, REST calls as you

Read top to bottom: Copilot only sees the tools you allow, then each call runs as you.



Headers limit the tools Copilot sees; your Azure DevOps permissions limit what each call returns.

When to build an MCP adapter, and when a CLI will do

Read left to right: when an adapter pays off, when it may not, and when a CLI is enough.

MCP ADAPTER PAYS OFF

Several compatible hosts need the same capability.

Backend is too broad for direct model access.

You need consistent schemas, bounds and citations.

A local API, COM object or folder needs a tool interface.

NOT AUTOMATICALLY BETTER

Your one application already owns reliable REST tools.

An extra proxy adds latency and operations.

Huge tool catalogs increase selection/context cost.

An existing supported connector already meets the need.

A CLI MAY BE ENOUGH

The agent already works in a terminal, like Copilot CLI.

A mature CLI exists, such as gh, az or kubectl.

People, scripts and CI already use the same commands.

Keep business logic in one reusable core; expose it through REST, MCP or a CLI as each host needs.



Azure DevOps MCP in the Copilot CLI and app.

Connect in Copilot

Connect Copilot to Azure DevOps

Use an existing provider server: work items, repositories, pipelines and wikis.

WHAT YOU CONNECT

Microsoft hosts and updates the remote ADO MCP server.

Streamable HTTP: no local Node process needed.

Use an Entra-backed ADO organization and an account with project access.

WHY IT IS USEFUL

Ask: List the projects in my organization.

Then: Summarize my open work items in Project X, with IDs and links.

Copilot retrieves current ADO data instead of guessing from the conversation.

Start with the remote server: nothing to install, and Copilot sees only what your account can access.

Add it in the Copilot CLI or App

Same provider endpoint; different host setup screens.

COPILOT CLI

Open /mcp in an interactive Copilot session and choose Add server.

Name: ado. Type: HTTP. Enter the organization endpoint.

Add read-only filtering, then complete provider sign-in if supported.

COPILOT APP

Customize > MCP: add a custom HTTP server using the same endpoint.

Under Installed, review status/tools and use Sign in when offered.

Start a session and try a small read-only ADO request.

Endpoint: https://mcp.dev.azure.com/{organization} | Replace {organization}; never paste a token.

The connection definition: URL, not local code

Copilot CLI/App config uses mcpServers; VS Code uses a different servers wrapper.

```
{
  "mcpServers": {
    "ado": {
      "type": "http",
      "url": "https://mcp.dev.azure.com/{organization}",
      "headers": {
        "X-MCP-Readonly": "true"
      }
    }
  }
}
```

WHAT THE FIELDS MEAN

ado: your local alias.

type: HTTP transport.

organization: your ADO organization, not a project name.

X-MCP-Readonly: asks ADO to expose only read operations.

User config: .copilot\mcp-config.json | Project config: .mcp.json or .github\mcp.json



Hello World first, then delegated sign-in for user data.

Build your own server

package.json

1 / SETUP: install dependencies in a new lab folder

```
{
  "name": "hello-mcp",
  "private": true,
  "type": "module",
  "engines": { "node": ">=22.18.0" },
  "scripts": {
    "start": "node server.ts",
    "test": "node client.ts"
  },
  "dependencies": {
    "@modelcontextprotocol/server": "2.0.0",
    "@modelcontextprotocol/client": "2.0.0",
    "zod": "4.5.4"
  }
}
```

1. Runtime and packages

Node 22.18+ runs TypeScript. ESM plus the official SDK and Zod.

2. Two commands

npm start serves MCP over stdio. npm test launches the real test client.

3. First run

Save all three files, then run npm install followed by npm test.

server.ts

2 / STARTUP: register hello; the handler runs only when the client calls it

```
import { McpServer } from '@modelcontextprotocol/server';
import { serveStdio } from '@modelcontextprotocol/server/stdio';
import * as z from 'zod/v4';

serveStdio(() => {
  const server = new McpServer({ name: 'hello-mcp', version: '1.0.0' });
  server.registerTool('hello', {
    description: 'Return a greeting for a supplied name.',
    inputSchema: z.object({ name: z.string().trim().min(1).max(80) })
  }, async ({ name }) => ({
    content: [{ type: 'text', text: `Hello, ${name}!` }]
  }));
  return server;
});
```

1. SDK does the plumbing

McpServer defines the server. serveStdio handles protocol messages.

2. name is the input

A nonempty name, at most 80 characters. Zod validates it before the handler.

3. Return a text result

hello({ name: 'World' }) returns Hello, World! No model or credentials needed.

client.ts

3 / TEST: launch server, discover hello, call it, check the response

```
import assert from 'node:assert/strict';
import { fileURLToPath } from 'node:url';
import { Client } from '@modelcontextprotocol/client';
import { StdioClientTransport } from '@modelcontextprotocol/client/stdio';

const client = new Client({ name: 'hello-test', version: '1.0.0' });
try {
  await client.connect(new StdioClientTransport({
    command: process.execPath,
    args: [fileURLToPath(new URL('./server.ts', import.meta.url))]
  }));
  const { tools } = await client.listTools();
  assert(tools.some(tool => tool.name === 'hello'));
  const result = await client.callTool({ name: 'hello', arguments: { name: 'World' } });
  assert(!result.isError);
  assert.deepEqual(result.content, [{ type: 'text', text: 'Hello, World!' }]);
  console.log('PASS: discovered hello and received Hello, World!');
} finally {
  await client.close();
}
```

1. Launch the server

process.execPath is Node. args resolves server.ts beside this client.

2. Discover and invoke

tools contains available tools. result contains the hello response.

3. Check and close

Assert the exact greeting. finally closes the transport even on failure.

Three identities, not one universal Copilot login

Left: a local stdio adapter. Right: a remote MCP server.

LOCAL STDIO DESIGN

1. Copilot account: access to the host.
2. OS user: launches your adapter.
3. Entra/Microsoft user: consents to Graph Notes.Read.

Use a delegated public-client flow and protected token cache; no client secret in source.

REMOTE MCP DESIGN

Client obtains an MCP-resource token.

Server validates its issuer, audience, expiry and scopes.

A separate supported delegated Graph flow supplies Graph access.

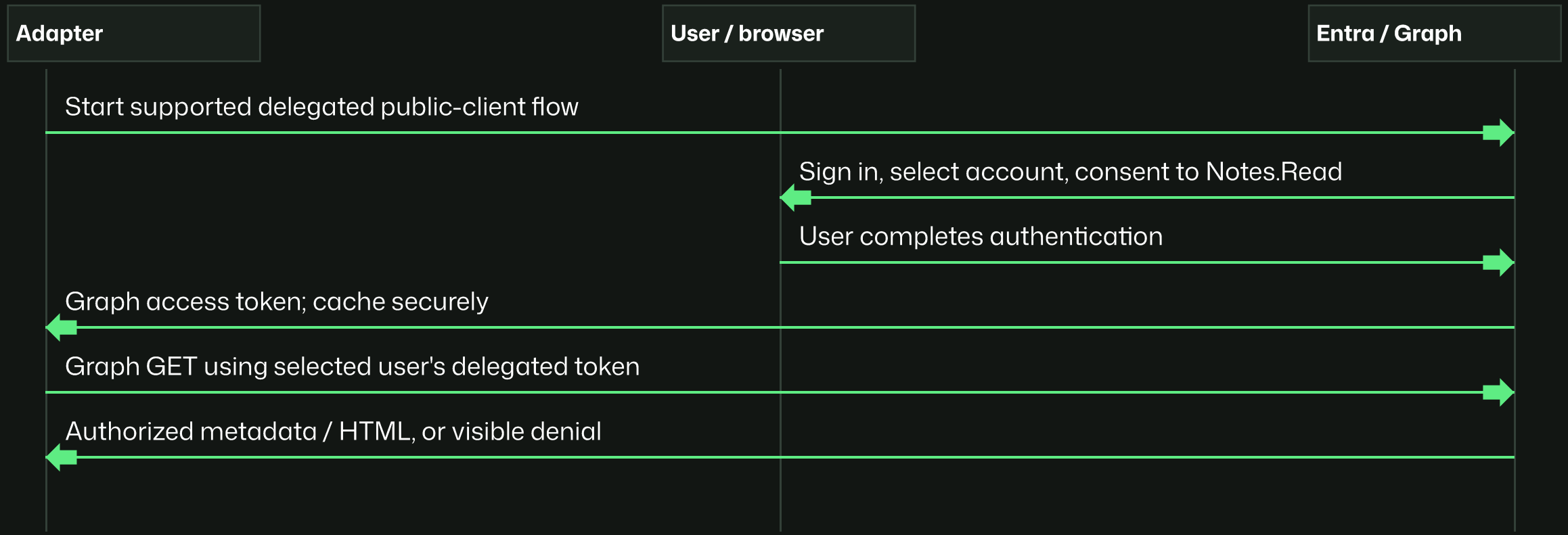
MCP token is not automatically a Graph token.

Never blindly forward it downstream.

Graph OneNote app-only authentication is unsupported effective 31 March 2025; use delegated access.

A local delegated flow has an explicit consent step

Read top to bottom: sign-in and consent happen before any Graph read.



Before auth is wired, fail with an actionable sign-in requirement; do not return empty notebooks.

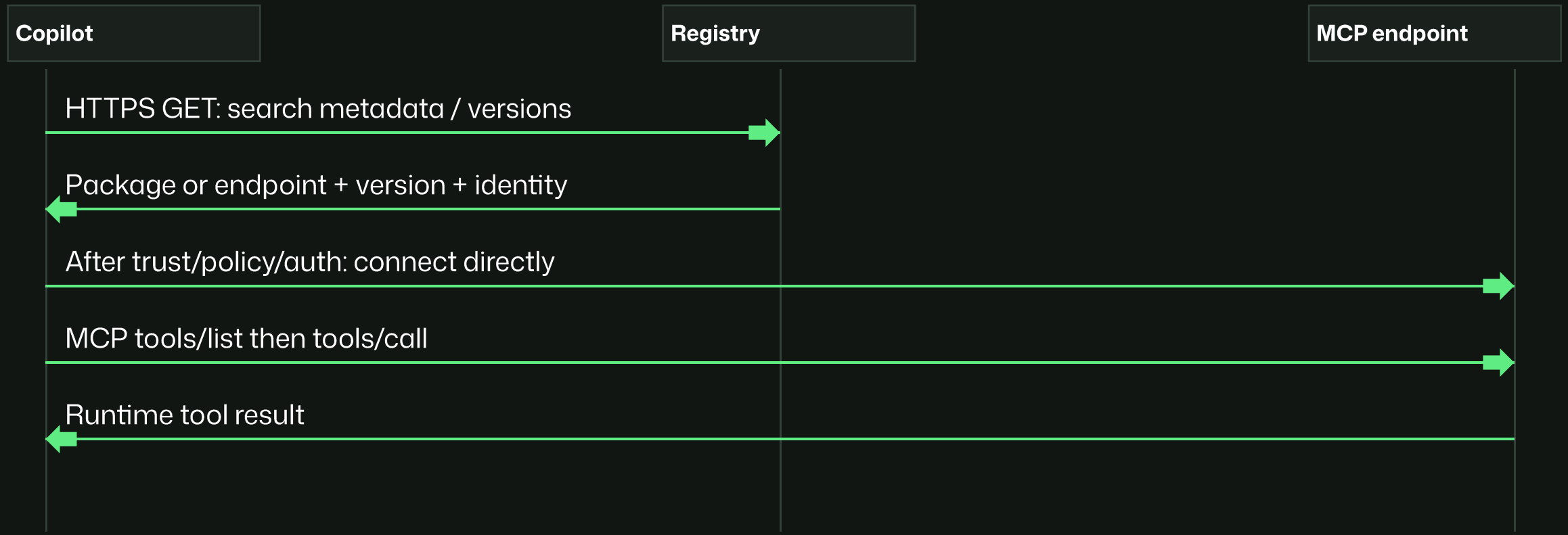


Discovery, versioned catalogs and execution controls.

Manage the registry

Registry discovery and runtime invocation are separate

Read top to bottom: the registry answers lookups; tool calls go straight to the server.



The registry is not a tool proxy, token store, data lake or notebook authorization service.

Query the starter registry from PowerShell

REGISTRY LAB / Start the catalog, then query from a second terminal

FIRST TERMINAL

```
npm run start:registry
# Local read-only registry: http://127.0.0.1:3000
# In a second terminal:
$base = 'http://127.0.0.1:3000'
$name = [uri]::EscapeDataString('com.example/mcp-setup-demo')
Invoke-RestMethod "$base/v0.1/servers?version=latest&limit=10"
Invoke-RestMethod "$base/v0.1/servers/$name/versions/latest"
Invoke-RestMethod "$base/v0.1/servers/$name/versions/1.0.0"
curl.exe -s -D - -o NUL "$base/v0.1/servers"
# Inspect CORS headers; registry writes return 405.
```

1. Registry process

start:registry serves read-only REST metadata; it does not execute MCP tools.

2. $base / $name

base is the registry origin. name encodes the server identity as one route segment.

3. Lookup and headers

List latest releases or fetch a version; curl exposes response/CORS headers.

The catalog's HTTP demo is separate from the txt-notes stdio tool; historic entries are not endorsements.

Discovery policy is not the strongest execution control

HOST POLICY / Evaluated when Copilot loads or uses MCP servers

MANAGED-SETTINGS.JSON / ILLUSTRATIVE ENTERPRISE POLICY

```
{
  "allowedMcpServers": [
    { "serverUrl": "https://mcp.example.com/onenote/mcp" }
  ],
  "deniedMcpServers": [
    { "serverUrl": "https://unreviewed.example.com/*" }
  ]
}
```

1. allowedMcpServers

Allow the approved runtime endpoint; this does not grant Graph permissions.

2. deniedMcpServers

Explicit deny takes precedence. With an allowlist, unmatched servers are blocked.

3. serverUrl pattern

Illustrative endpoint rule. Policy lives in managed-settings.json, not the catalog.

GitHub recommends enterprise managed settings; registry-based restriction remains a preview mechanism.

Connect first, build where needed, govern always

Read left to right: three moves, in the order to make them.

1. CONNECT FIRST

Add the remote Azure DevOps MCP server.

Start read-only; widen access only when needed.

Ask for live data: projects, work items, PRs.

See section 04

2. BUILD WHERE NEEDED

Wrap one REST capability as a narrow, typed tool.

Act with the signed-in user's delegated access.

Return bounded results with links to the source.

See sections 03 and 05

3. GOVERN ALWAYS

Publish approved servers in your MCP registry.

Enforce allow and deny lists in managed settings.

Remember: the registry guides, policy enforces.

See section 06

Start read-only with a provider server; build and govern only what your teams need.

Sources and starter code

PROTOCOL AND SDK

Protocol versioning and compatibility
https://modelcontextprotocol.io/specification/2026-07-28/basic/versioning

JSON-RPC, metadata and statelessness
https://modelcontextprotocol.io/specification/2026-07-28/basic/index

Tool schemas, results and errors
https://modelcontextprotocol.io/specification/2026-07-28/server/tools

Streamable HTTP transport
https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http

Official TypeScript SDK v2
https://ts.sdk.modelcontextprotocol.io/v2/

Client execution and error semantics
https://ts.sdk.modelcontextprotocol.io/v2/get-started/first-client.html

COPILOT, AZURE DEVOPS AND GOVERNANCE

Azure DevOps remote MCP server
https://learn.microsoft.com/en-us/azure/devops/mcp-server/remote-mcp-server

Copilot CLI setup and App customization
https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/add-mcp-servers

Copilot registry requirements
https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-mcp-usage/configure-mcp-registry

Enterprise execution allowlists
https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-mcp-usage/configure-enterprise-allowlist

Runnable starter source
https://github.com/melabadi/mcp-starter/tree/49c88ab242f9fc9c957280d3bd8ba9f8a9a350ba

OneNote overview: delegated auth requirement
https://learn.microsoft.com/en-us/graph/integrate-with-onenote